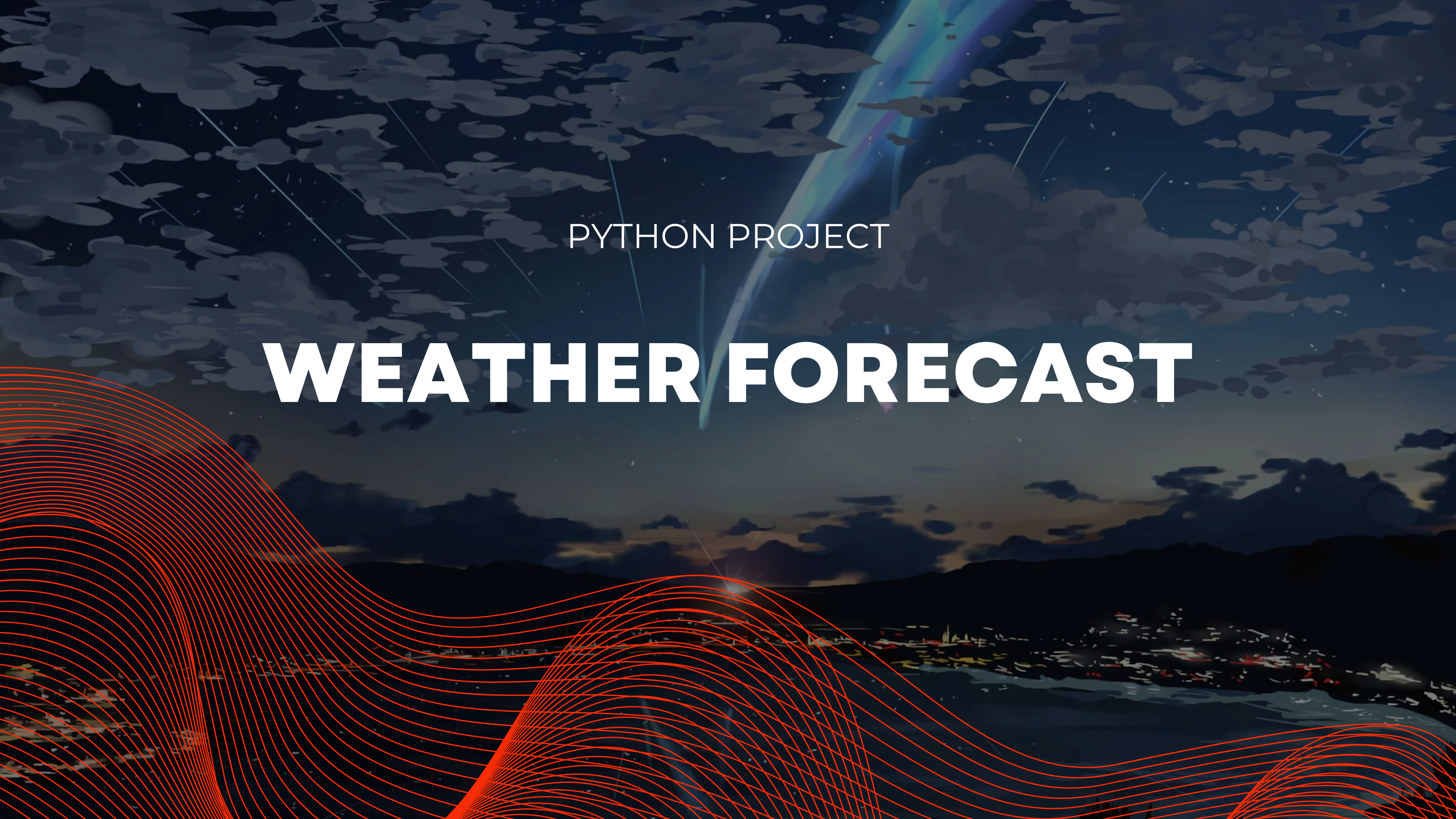




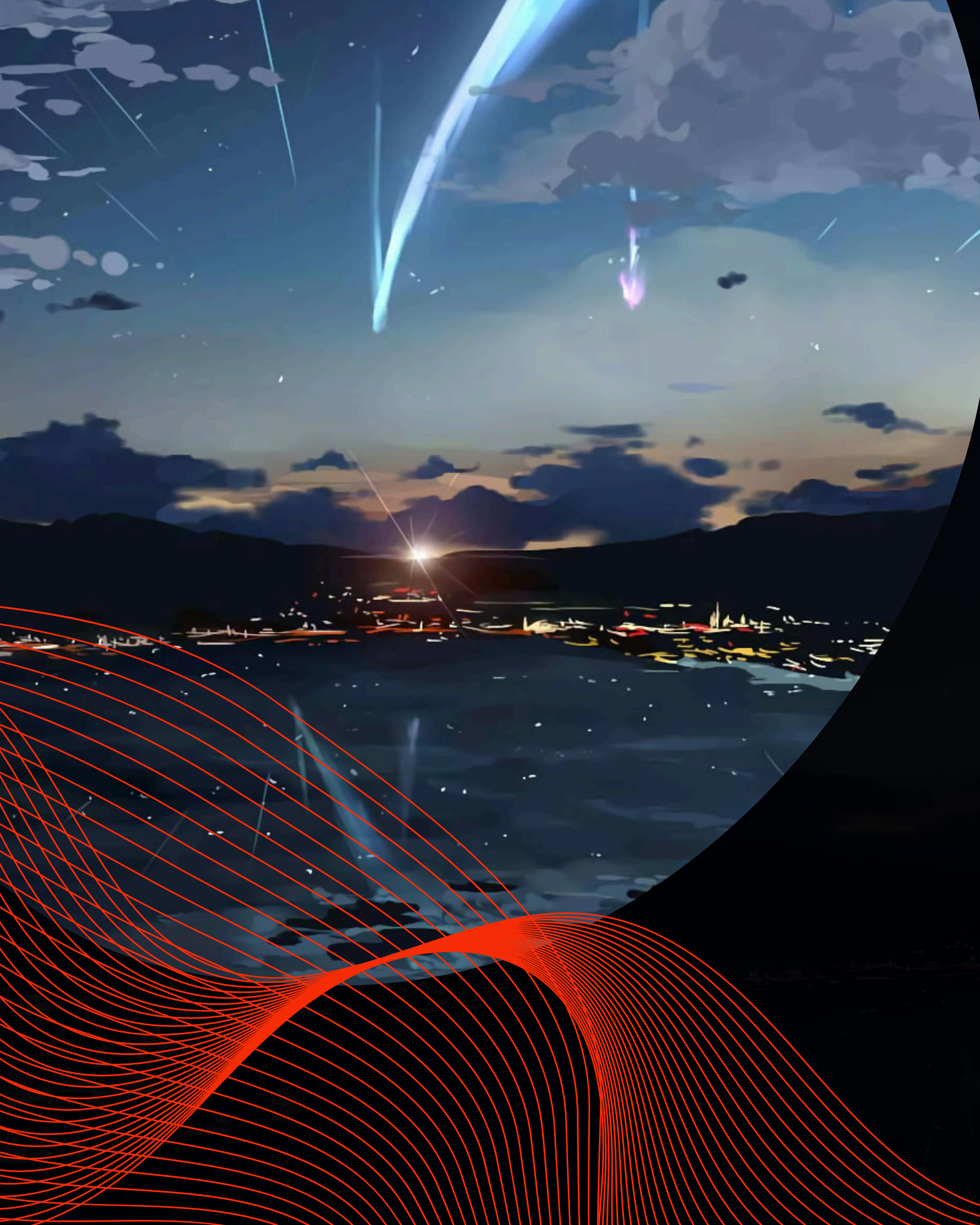
PYTHON PROJECT

WEATHER FORECAST

CONTENT

- 1 Introduction
- 2 Technologies Used
- 3 Project Structure
- 4 GUI Design
- 5 Fetching Weather Data

- 6 Error Handling
- 7 Icon Management
- 8 Code Snippets
- 9 Fetching Weather Data
- 10 Thank You

The background of the slide features a circular inset showing a night scene of a city with lights reflecting on water, under a dark sky with several bright, colorful meteor streaks. The rest of the slide has a dark, textured background with a series of concentric, wavy orange lines in the bottom left corner.

INTRODUCTION



Overview

- WeatherOcast is a Python-based application designed to provide real-time weather updates.
- Users can enter a city name to retrieve current weather conditions and forecasts.



Purpose

- To create a user-friendly interface for accessing weather information quickly and easily.

TECHNOLOGIES USED

Programming Language:

PYTHON

Libraries:

- CustomTkinter: For a modern and customizable graphical user interface.
- Requests: To handle HTTP requests for API calls.
- Pillow (PIL): For image processing and handling weather icons.

API:

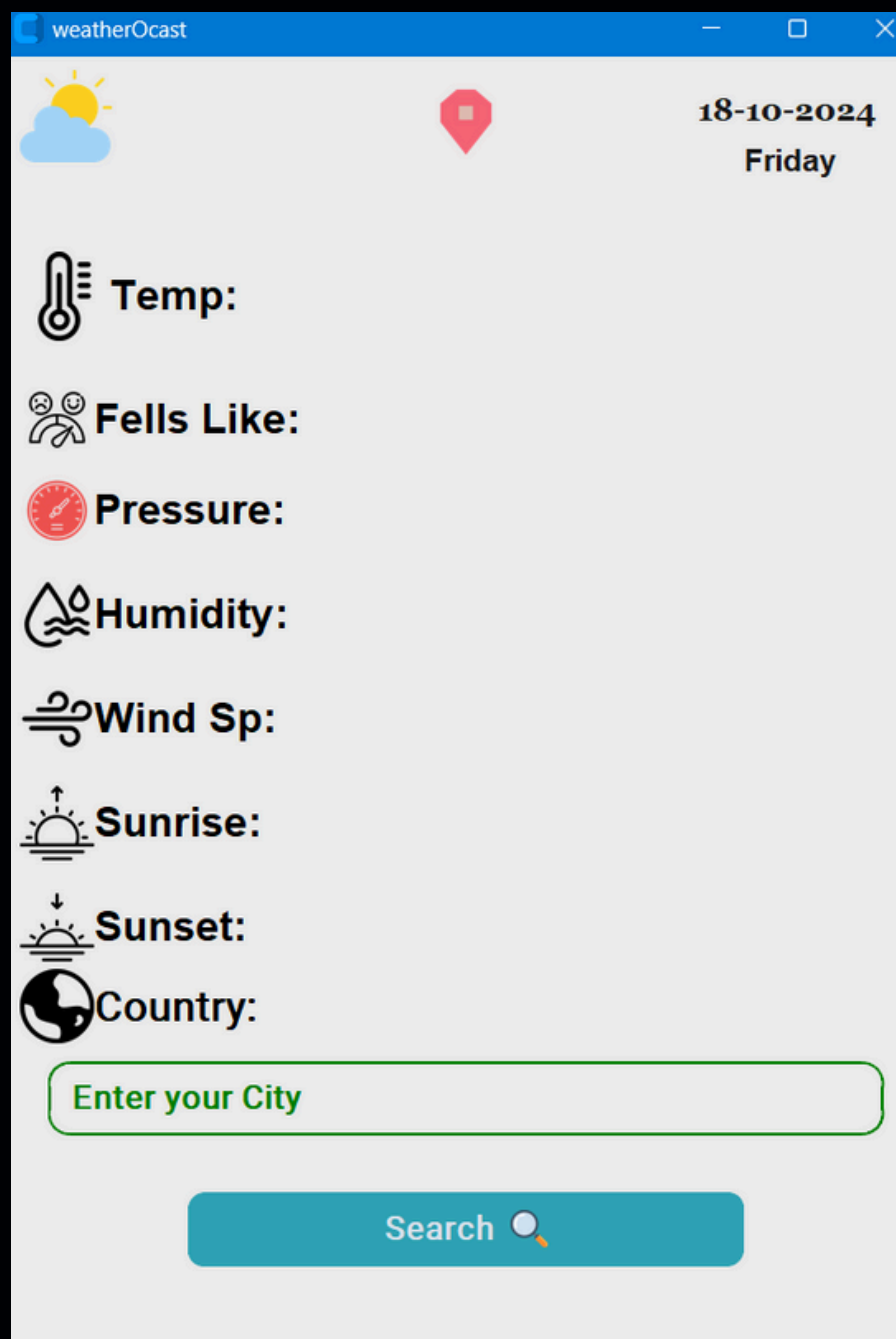
- OpenWeatherMap: Provides weather data through a robust API.

PROJECT STRUCTURE

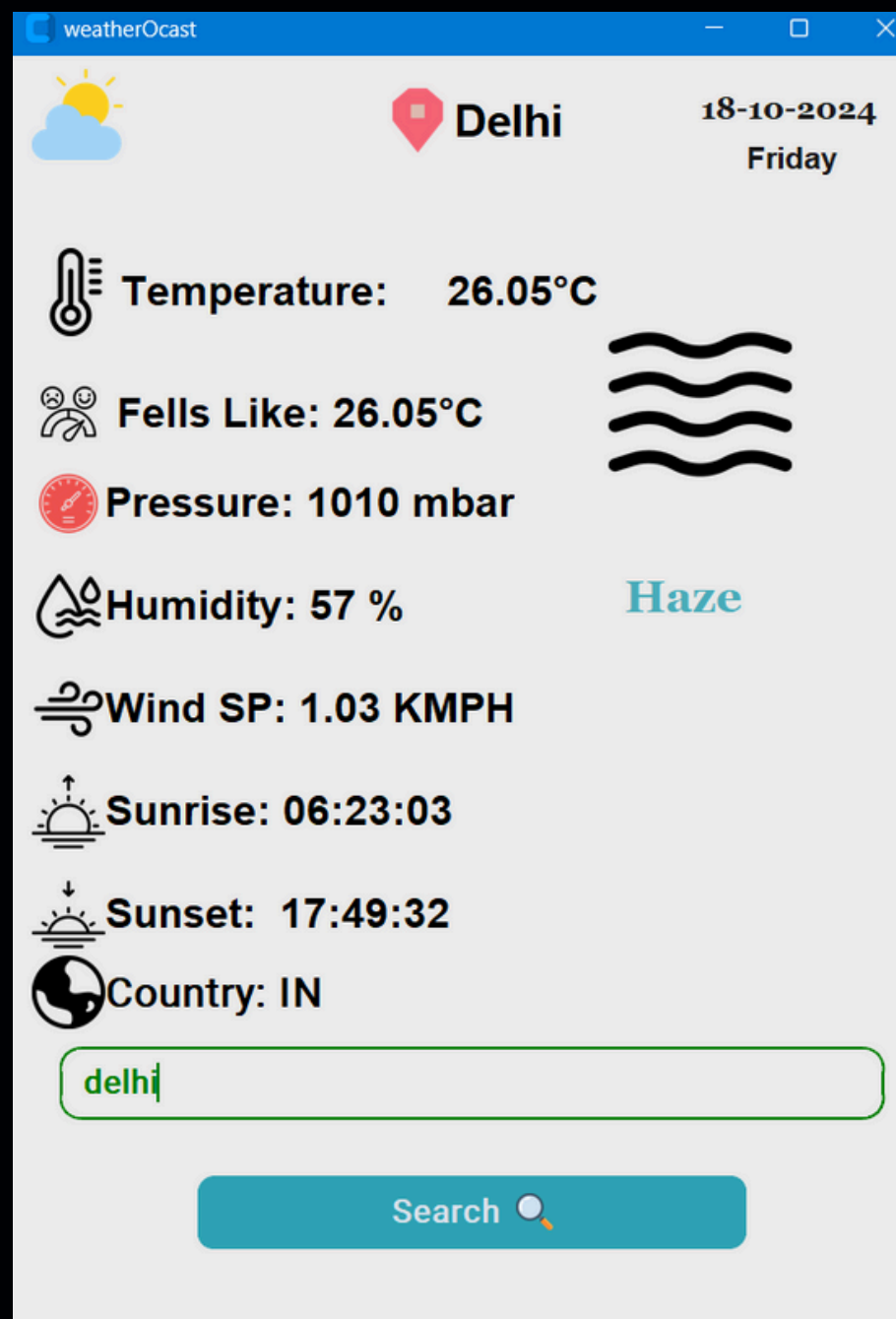
Main Components:

- GUI Setup: Utilizing CustomTkinter to create the application layout.
- API Integration: Fetching weather data based on user input.
- Error Handling: Ensuring a smooth user experience by managing potential errors.





Home Page



Running Code

GUI DESIGN

Layout:

- City Input Field: Where users type in the name of the city.
- Display Area: Shows weather data, including temperature, humidity, etc.
- Search Button: Triggers the weather data retrieval process.

Dynamic Elements:

- Weather icons change based on current conditions for visual representation.

FETCHING WEATHER DATA

Function: `get_weather()`

- Description: Responsible for retrieving weather data from the OpenWeatherMap API.
- Process:
 - a. Get city name: Retrieves user input from the entry field.
 - b. API Request: Constructs the URL and makes a GET request.
 - c. JSON Parsing: Converts the API response into a usable format.
 - d. GUI Update: Displays the retrieved weather information in the app.



ERROR HANDLING

Types of Errors Managed:

- Invalid City Names: Alerts the user if the city name is not recognized.
- Network Issues: Handles exceptions for network-related errors.
- User Feedback:
- Utilizes message boxes to inform users about errors in a clear and friendly manner.

ICON MANAGEMENT

Purpose:

- Icons play a critical role in enhancing user experience by providing immediate visual context.

Usage:

- Weather conditions are represented through specific icons (e.g., sun for clear, clouds for overcast, raindrops for rain).
- Icons accompany labels for metrics like temperature, humidity, and wind speed, making the interface intuitive.

Implementation:

- Utilized the CTkImage class from the customtkinter library to manage icon display.
- Icons are loaded from local files and resized appropriately for the GUI layout.



CODE SNIPPETS

Fetching Weather Data:

- The function that makes the API call to retrieve weather data based on user input.
- Uses the requests library to handle HTTP requests.

```
def get_weather(*args):  
    city_name_ = city_name.get()  
    url = f"http://api.openweathermap.org/data/2.5/weather?q={city_name_}&appid={api_key}&units=metric"  
    response = requests.get(url)  
    weather_data = response.json()
```

Uses the requests library to call the OpenWeatherMap API.

```
response = requests.get(url)  
weather_data = response.json()
```

FETCHING WEATHER DATA

Process:

- When the user enters a city name and hits "Search," the application triggers the `get_weather` function.

Data Handling:

- The function checks the response code to ensure valid data retrieval.
- Extracts essential weather information such as temperature, pressure, humidity, and more from the response.

Display:

- The UI is updated dynamically with the fetched data, ensuring users have the latest weather information.

```
if weather_data["cod"] == 200:  
    temp = weather_data["main"]["temp"]
```

← checks if the API response indicates a successful request.

GITHUB REPOSITORY



Feel free to contribute to the project by opening issues or submitting pull requests!

<https://github.com/YashGarg1122/WeatherForecast.git>

TEAM CONTRIBUTIONS

YASH GARG

Set up API,
interface design

ANSHUL SIROHI

Documentation,
UI Design

DHRUV GUPTA

Lead developer,
account
management

SAURAV AGRAWAL

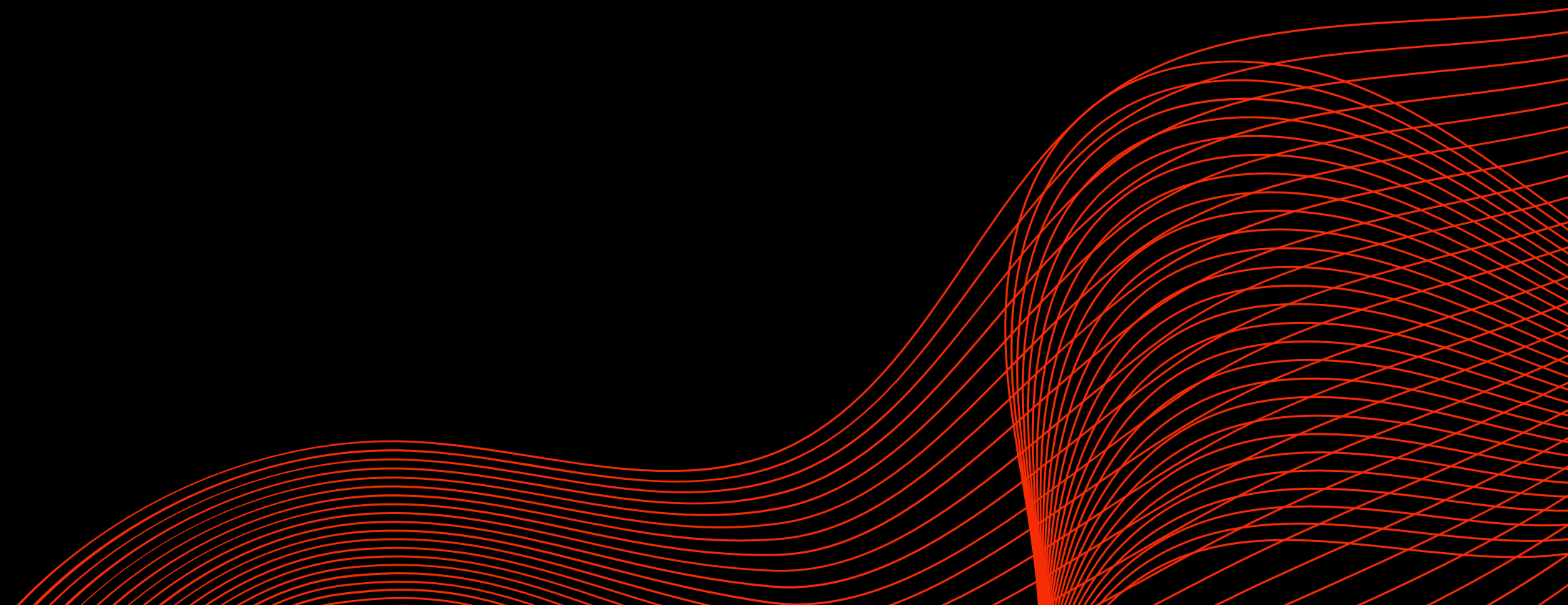
Git
management,
database setup

APURVA BANSAL

Project
management

RITESH KUMAR SINHA

Error handling,
testing,
database
design



The background features a series of thin, flowing orange lines that create a sense of movement and depth against a dark blue background. The lines are concentrated in the lower half of the image, forming a series of peaks and valleys that resemble a stylized landscape or a series of waves. The text "THANK YOU" is centered horizontally and partially overlaps the flowing lines.

THANK YOU